Практическая работа №4

Интеграция Flask и SQLite (чтение данных)

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Подключить базу данных SQLite к Flask-приложению и вывести список задач на веб-страницу с помощью шаблонизатора Jinja2.

Технологии: Python 3; Flask; шаблонизатор Jinja2 (render_template); sqlite3, sqlite3.Row; HTML, CSS.

Реализация: При запросе GET / приложение открывает соединение, выполняет SELECT id, title, status FROM tasks, закрывает соединение и вызывает render_template('index.html', items=rows). Благодаря sqlite3.Row в шаблоне доступны поля по имени: item.title, item.status. Выполненные задачи отображаются зачёркнутыми, при пустой базе выводится «Список пуст».

Основные модули:

- `src/app.py` — функция get_connection() (соединение + row_factory = sqlite3.Row), маршрут / выполняет SELECT и передаёт строки в шаблон.
- `src/templates/index.html` — шаблон Jinja2: цикл {% for %} по задачам, условие {% if %} для пустого списка и CSS-класс done для выполненных задач.

Ход работы

Шаг 1. Подготовка структуры

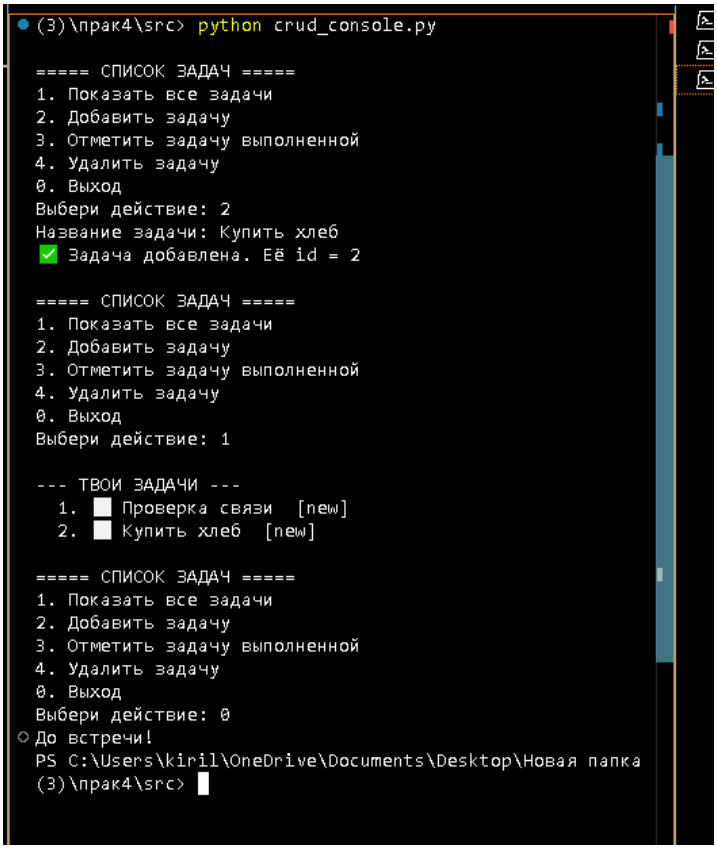


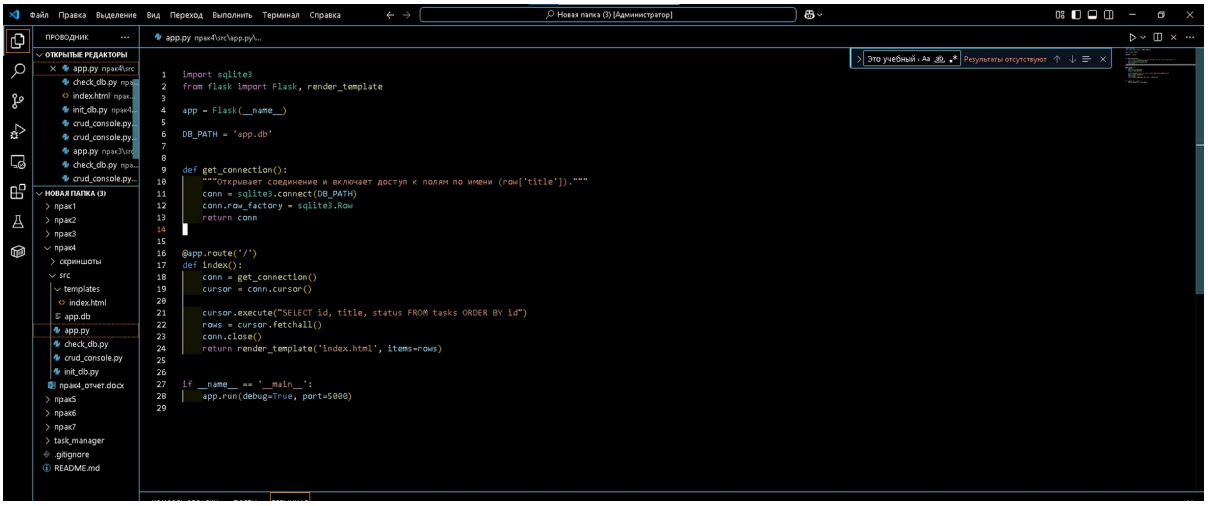
Рисунок 1 — добавление задач в базу через crud_console.py

Шаг 2. Создание шаблона templates/index.html



Рисунок 2 — шаблон templates/index.html: цикл for и условие if на Jinja2

Шаг 3. Переписываем app.py: чтение из базы



```python
import sqlite3
from flask import Flask, render_template

app = Flask(__name__)

DB_PATH = 'app.db'


def get_connection():
    """Открывает соединение и включает доступ к полям по имени (row['title'])."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn


@app.route('/')
def index():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT id, title, status FROM tasks ORDER BY id")
    rows = cursor.fetchall()
    conn.close()
    return render_template('index.html', items=rows)


if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Рисунок 3 — app.py: подключение к базе с sqlite3.Row и передача задач в шаблон

Шаг 4. Запуск и проверка

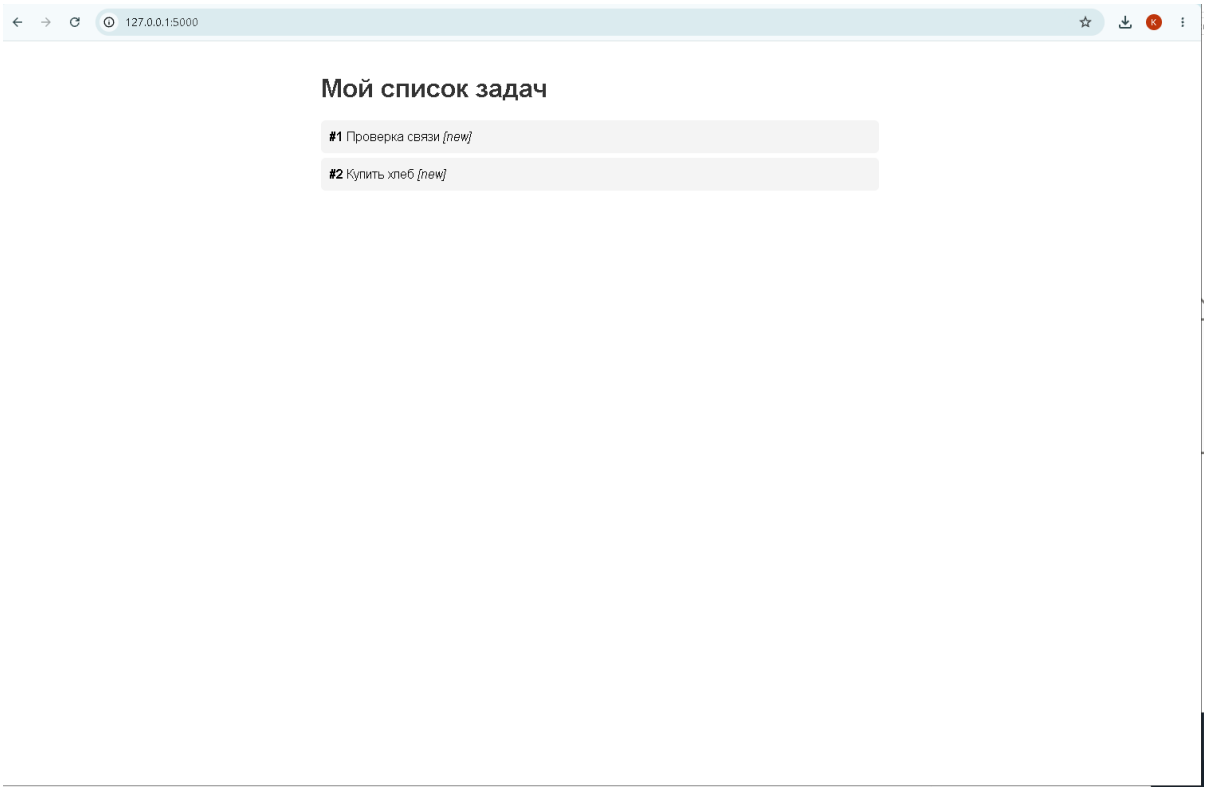


Рисунок 4 — страница http://127.0.0.1:5000 со списком задач из базы

Шаг 5. Проверка «живой» связи с базой

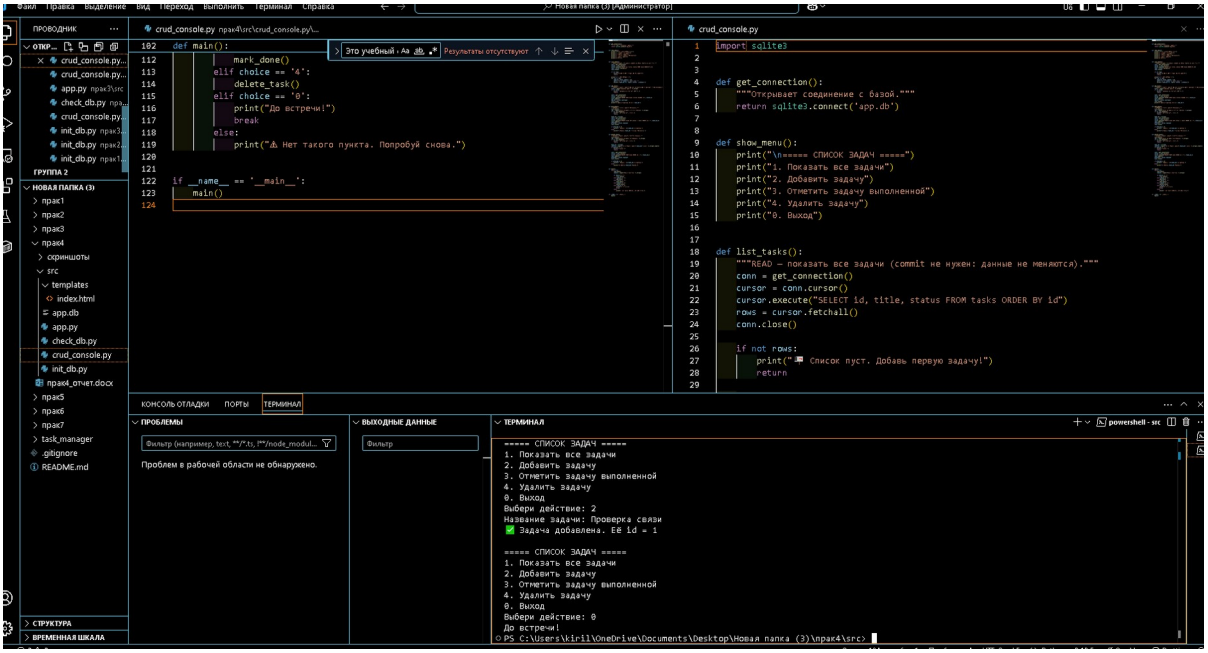


Рисунок 5 — добавление задачи «Проверка связи» через crud_console.py во втором терминале

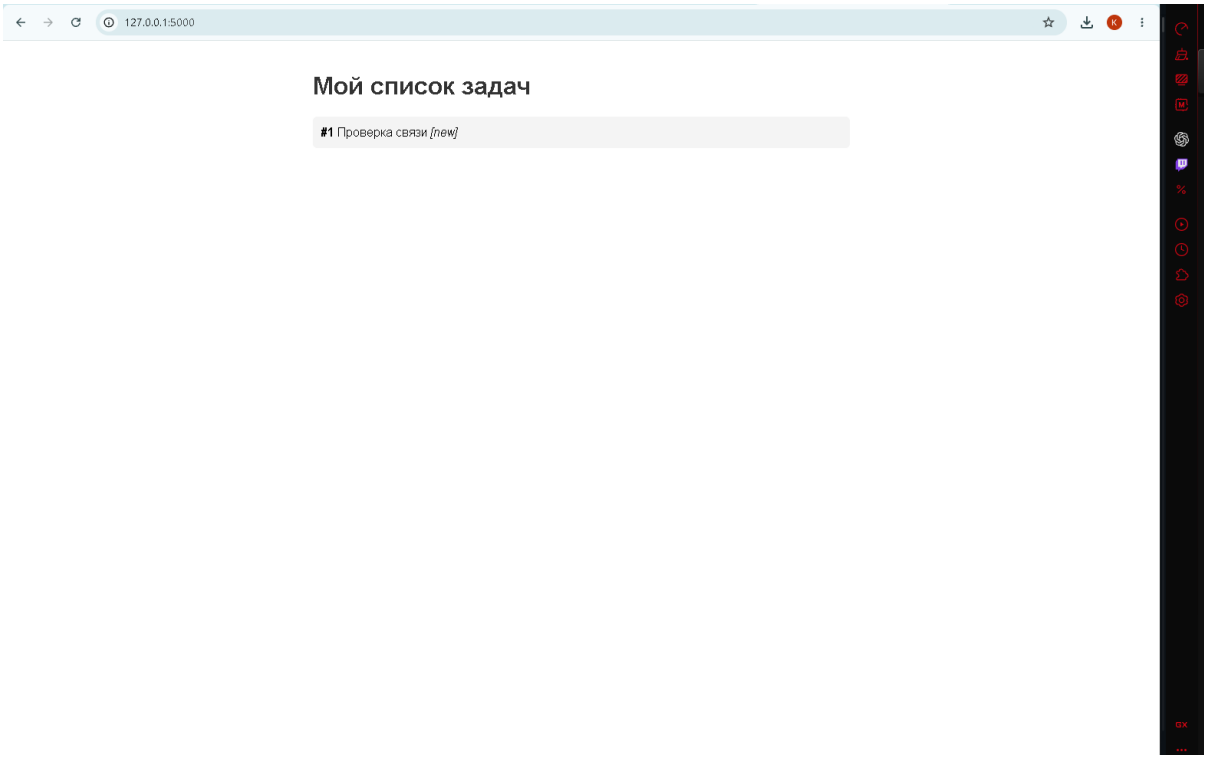


Рисунок 6 — задача из базы появилась на веб-странице после обновления

Шаг 6. Проверка пустой базы

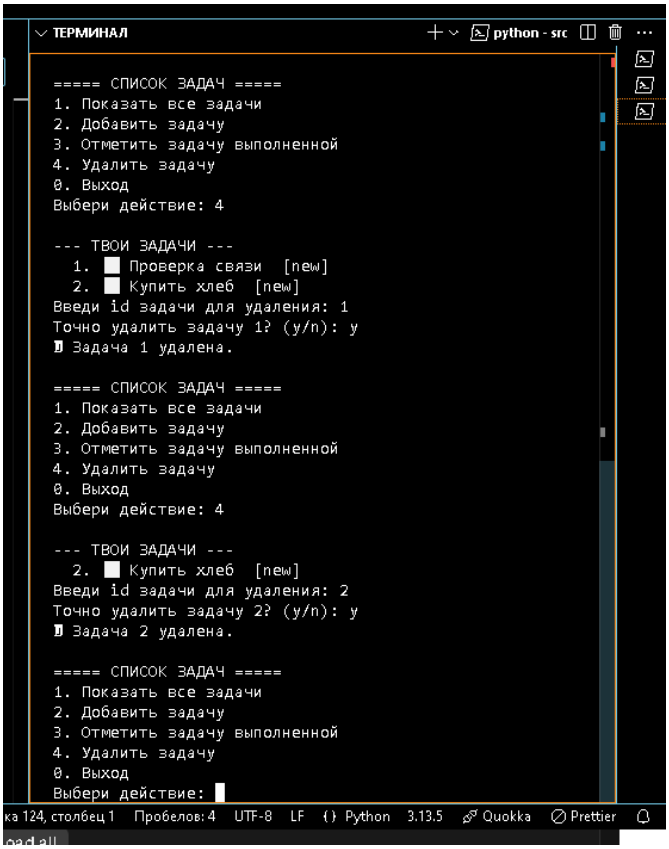


Рисунок 7 — удаление всех задач через crud_console.py

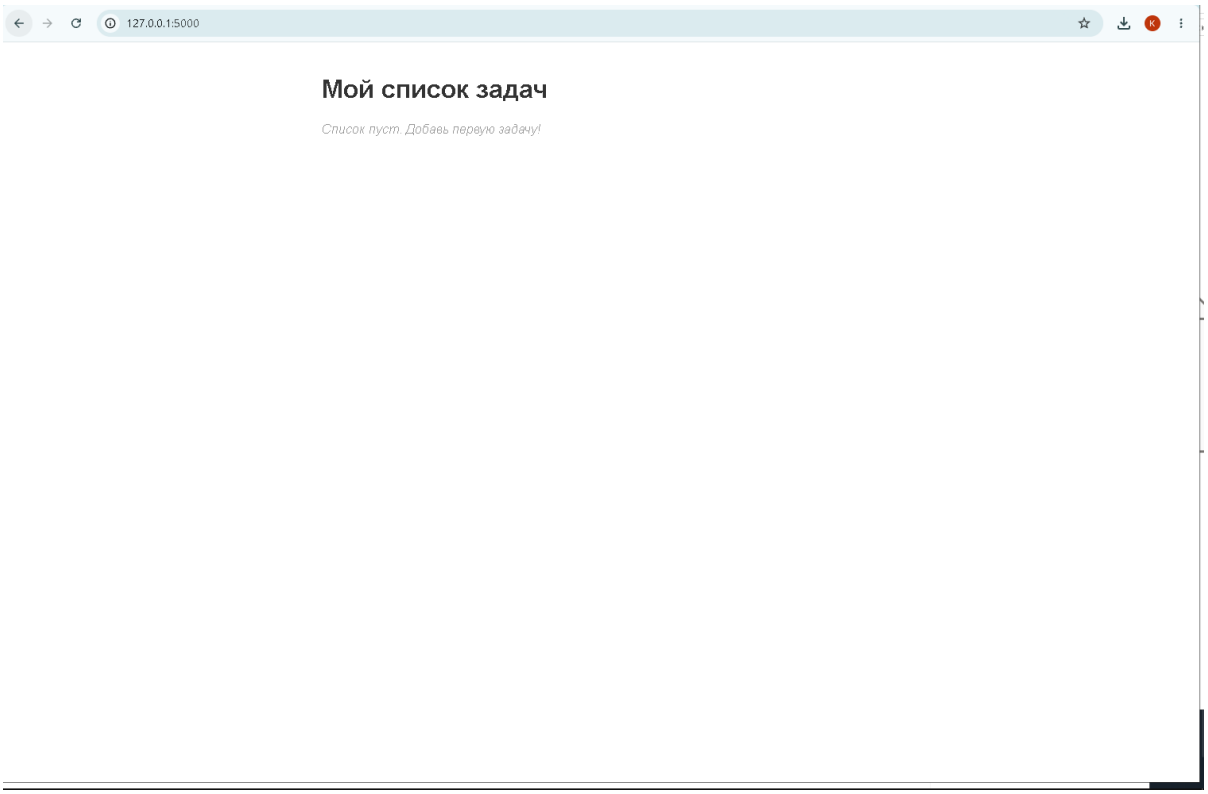


Рисунок 8 — пустая база — на странице сообщение «Список пуст. Добавь первую задачу!»

Примеры запросов и ответов

```
GET / → 200 OK, HTML-страница со списком задач из app.db
#1 Купить хлеб [new]
#2 Сделать уроки [done]   (зачёркнуто)
```

Ответы на контрольные вопросы

1. **Зачем нужен Jinja2, если можно вернуть HTML-строку?**

Шаблон отделяет HTML от Python-кода: его удобнее читать и править, можно использовать циклы и условия, переиспользовать части страницы, а данные автоматически экранируются.

2. **Чем {{ ... }} отличается от {% ... %}?**

{{ ... }} выводит значение на страницу; {% ... %} — управляющая конструкция (цикл for, условие if), сама по себе ничего не выводит.

3. **Что делает render_template('index.html', items=rows)?**

Находит файл templates/index.html, подставляет в него переменную items со списком строк и возвращает готовый HTML.

4. **Почему при чтении данных не нужен conn.commit()?**

SELECT не изменяет данные, фиксировать нечего.

5. **Что такое sqlite3.Row и почему item.title работает только с ним?**

Это тип строки результата, у которого можно обращаться к полям по имени колонки. Обычный кортеж поддерживает только индексы (item[1]), поэтому item.title с ним не работает.

6. **Куда Flask кладёт шаблоны по умолчанию и можно ли это изменить?**

В папку templates рядом с файлом приложения. Изменить можно параметром Flask(__name__, template_folder='другая_папка').

7. **Что выведет шаблон, если items — пустой список?**

Сработает ветка {% else %} — сообщение «Список пуст. Добавь первую задачу!».

Вывод

Flask-приложение подключено к базе SQLite, данные выводятся на страницу через шаблон Jinja2. Проверено, что страница отображает актуальные данные из общего файла app.db и корректно обрабатывает пустой список.